Assignment 5

Database Creation and Table Implementation

Creating the LibraryManagementSystem database and implementing all tables with SQL DDL

Project: Library Management System **Name:** _____ **Roll No:** _____

1. Objective

This assignment creates the **LibraryManagementSystem** database and implements, with SQL DDL commands, all nine tables of the finalized design: *Authors, Publishers, Categories, Books, Members, Staff, Loans, Reservations* and *Fines*. Primary keys, foreign keys and the relationships between tables are defined in the script. Every command in this document was executed on a live MySQL-compatible server (MariaDB 10.11) and the outputs shown are the real results.

2. System Overview and Table Summary

The system manages a library: it catalogues books (with their author, publisher and category), registers members, records which staff member issued which book to which member, handles reservations for books that are not currently available, and records fines for late returns.

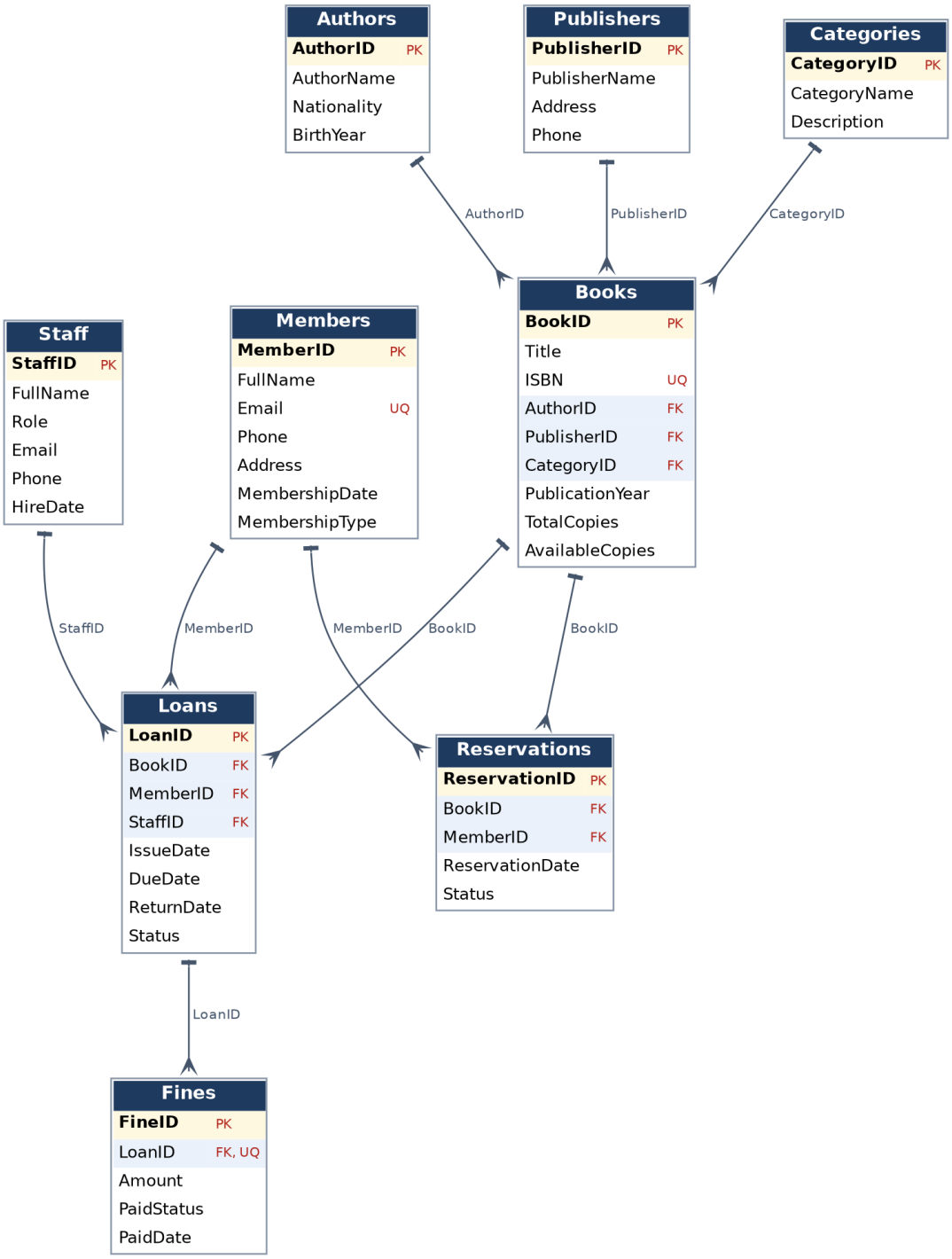
#	Table	Purpose	Primary key	Foreign keys
1	Authors	Writers of the books	AuthorID	-
2	Publishers	Publishing houses	PublisherID	-
3	Categories	Subject / genre classification	CategoryID	-
4	Books	Catalogue of titles with stock counts	BookID	AuthorID, PublisherID, CategoryID
5	Members	Registered library members	MemberID	-
6	Staff	Library employees	StaffID	-
7	Loans	Book issue / return transactions	LoanID	BookID, MemberID, StaffID
8	Reservations	Reservation requests for books	ReservationID	BookID, MemberID
9	Fines	Fines charged for overdue loans	FineID	LoanID (also UNIQUE)

3. Relationships and Cardinality

Parent (one)	Child (many)	Foreign key	Meaning
Authors	Books	Books.AuthorID	One author writes many books
Publishers	Books	Books.PublisherID	One publisher publishes many books
Categories	Books	Books.CategoryID	One category contains many books
Books	Loans	Loans.BookID	A book can be loaned many times
Members	Loans	Loans.MemberID	A member can borrow many times
Staff	Loans	Loans.StaffID	A staff member issues many loans
Books	Reservations	Reservations.BookID	A book can be reserved many times
Members	Reservations	Reservations.MemberID	A member can make many reservations
Loans	Fines	Fines.LoanID (UNIQUE)	A loan has at most one fine (one-to-one)

4. Entity-Relationship (ER) Diagram

The diagram below was generated from the implemented design. Each arrow runs from the **child** table's foreign key to the **parent** table's primary key; the crow's-foot end is the "many" side. PK = primary key, FK = foreign key, UQ = unique.



5. SQL DDL Script

The complete script is given below. Tables are created in dependency order: the independent (parent) tables first (*Authors, Publishers, Categories, Members, Staff*), then the tables that reference them (*Books, Loans, Reservations, Fines*). Creating a child table before its parent would fail because the referenced table would not yet exist.

```sql
-- DBMS Assignment: Library Management System
-- Based on the submitted database design
-- 9 Tables: Authors, Publishers, Categories, Books, Members,
-- Staff, Loans, Reservations, Fines

CREATE DATABASE IF NOT EXISTS LibraryManagementSystem;
USE LibraryManagementSystem;

-- 1. Authors
CREATE TABLE Authors (
    AuthorID INT PRIMARY KEY,
    AuthorName VARCHAR(100) NOT NULL,
    Nationality VARCHAR(50),
    BirthYear INT
);

-- 2. Publishers
CREATE TABLE Publishers (
    PublisherID INT PRIMARY KEY,
    PublisherName VARCHAR(100) NOT NULL,
    Address VARCHAR(150),
    Phone VARCHAR(15)
);

-- 3. Categories
CREATE TABLE Categories (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL,
    Description VARCHAR(200)
);

-- 4. Books
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(150) NOT NULL,
    ISBN VARCHAR(20) UNIQUE,
    AuthorID INT,
    PublisherID INT,
    CategoryID INT,
    PublicationYear INT,
    TotalCopies INT NOT NULL,
    AvailableCopies INT NOT NULL,
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID),
    FOREIGN KEY (PublisherID) REFERENCES Publishers(PublisherID),
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

-- 5. Members
CREATE TABLE Members (
    MemberID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(150),
    MembershipDate DATE NOT NULL,
    MembershipType VARCHAR(20)
);

-- 6. Staff
CREATE TABLE Staff (
    StaffID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Role VARCHAR(50),
    Email VARCHAR(100),
    Phone VARCHAR(15),
    HireDate DATE
);

-- 7. Loans
CREATE TABLE Loans (
    LoanID INT PRIMARY KEY,
    BookID INT,
    MemberID INT,
    StaffID INT,
    IssueDate DATE NOT NULL,
    DueDate DATE NOT NULL,
    ReturnDate DATE,
    Status VARCHAR(20),
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID),
```

```
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID)
);

-- 8. Reservations
CREATE TABLE Reservations (
    ReservationID INT PRIMARY KEY,
    BookID INT,
    MemberID INT,
    ReservationDate DATE NOT NULL,
    Status VARCHAR(20),
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);

-- 9. Fines
CREATE TABLE Fines (
    FineID INT PRIMARY KEY,
    LoanID INT UNIQUE,
    Amount DECIMAL(6,2) NOT NULL,
    PaidStatus VARCHAR(10),
    PaidDate DATE,
    FOREIGN KEY (LoanID) REFERENCES Loans(LoanID)
);

-- End of Library Management System Database
```

Execution log

Statement (first line)	Result
CREATE DATABASE IF NOT EXISTS LibraryManagementSystem	Query OK
USE LibraryManagementSystem	Query OK
CREATE TABLE Authors (	Query OK
CREATE TABLE Publishers (	Query OK
CREATE TABLE Categories (	Query OK
CREATE TABLE Books (	Query OK
CREATE TABLE Members (	Query OK
CREATE TABLE Staff (	Query OK
CREATE TABLE Loans (	Query OK
CREATE TABLE Reservations (	Query OK
CREATE TABLE Fines (	Query OK

> **All 11 statements executed successfully** (1 CREATE DATABASE, 1 USE, 9 CREATE TABLE) with no errors.

6. Verification of the Implementation

Step 1: confirm that the database exists

```
SHOW DATABASES LIKE 'LibraryManagementSystem';
```

Output: (1 row)

Database (LibraryManagementSystem)
LibraryManagementSystem

Step 2: list the tables created in the database

```
SHOW TABLES;
```

Output: (9 rows)

Tables_in_LibraryManagementSystem
Authors
Books
Categories
Fines
Loans
Members
Publishers
Reservations
Staff

Explanation: Exactly the nine tables of the design are present.

Step 3: table structures (DESCRIBE)

DESCRIBE shows each column, its data type, whether NULL is allowed, the key type (PRI = primary key, UNI = unique, MUL = non-unique index, which MySQL creates automatically for foreign keys), and any default value.

DESCRIBE Authors

```
DESCRIBE Authors;
```

Output: (4 rows)

Field	Type	Null	Key	Default	Extra
AuthorID	int(11)	NO	PRI	NULL	
AuthorName	varchar(100)	NO		NULL	
Nationality	varchar(50)	YES		NULL	
BirthYear	int(11)	YES		NULL	

DESCRIBE Publishers

```
DESCRIBE Publishers;
```

Output: (4 rows)

Field	Type	Null	Key	Default	Extra
PublisherID	int(11)	NO	PRI	NULL	
PublisherName	varchar(100)	NO		NULL	
Address	varchar(150)	YES		NULL	
Phone	varchar(15)	YES		NULL	

DESCRIBE Categories

```
DESCRIBE Categories;
```

Output: (3 rows)

Field	Type	Null	Key	Default	Extra
CategoryID	int(11)	NO	PRI	NULL	
CategoryName	varchar(50)	NO		NULL	
Description	varchar(200)	YES		NULL	

DESCRIBE Books

```
DESCRIBE Books;
```

Output: (9 rows)

Field	Type	Null	Key	Default	Extra
BookID	int(11)	NO	PRI	NULL	
Title	varchar(150)	NO		NULL	
ISBN	varchar(20)	YES	UNI	NULL	
AuthorID	int(11)	YES	MUL	NULL	
PublisherID	int(11)	YES	MUL	NULL	
CategoryID	int(11)	YES	MUL	NULL	
PublicationYear	int(11)	YES		NULL	
TotalCopies	int(11)	NO		NULL	
AvailableCopies	int(11)	NO		NULL	

DESCRIBE Members

```
DESCRIBE Members;
```

Output: (7 rows)

Field	Type	Null	Key	Default	Extra
MemberID	int(11)	NO	PRI	NULL	
FullName	varchar(100)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Phone	varchar(15)	YES		NULL	
Address	varchar(150)	YES		NULL	
MembershipDate	date	NO		NULL	
MembershipType	varchar(20)	YES		NULL	

DESCRIBE Staff

```
DESCRIBE Staff;
```

Output: (6 rows)

Field	Type	Null	Key	Default	Extra
StaffID	int(11)	NO	PRI	NULL	
FullName	varchar(100)	NO		NULL	
Role	varchar(50)	YES		NULL	
Email	varchar(100)	YES		NULL	
Phone	varchar(15)	YES		NULL	
HireDate	date	YES		NULL	

DESCRIBE Loans

```
DESCRIBE Loans;
```

Output: (8 rows)

Field	Type	Null	Key	Default	Extra
LoanID	int(11)	NO	PRI	NULL	
BookID	int(11)	YES	MUL	NULL	
MemberID	int(11)	YES	MUL	NULL	
StaffID	int(11)	YES	MUL	NULL	
IssueDate	date	NO		NULL	
DueDate	date	NO		NULL	
ReturnDate	date	YES		NULL	
Status	varchar(20)	YES		NULL	

DESCRIBE Reservations

```
DESCRIBE Reservations;
```

Output: (5 rows)

Field	Type	Null	Key	Default	Extra
ReservationID	int(11)	NO	PRI	NULL	
BookID	int(11)	YES	MUL	NULL	
MemberID	int(11)	YES	MUL	NULL	
ReservationDate	date	NO		NULL	
Status	varchar(20)	YES		NULL	

DESCRIBE Fines

```
DESCRIBE Fines;
```

Output: (5 rows)

Field	Type	Null	Key	Default	Extra
FineID	int(11)	NO	PRI	NULL	
LoanID	int(11)	YES	UNI	NULL	
Amount	decimal(6,2)	NO		NULL	
PaidStatus	varchar(10)	YES		NULL	
PaidDate	date	YES		NULL	

Step 4: confirm that all foreign-key relationships are defined

```
SELECT TABLE_NAME AS child_table, COLUMN_NAME AS fk_column,
       REFERENCED_TABLE_NAME AS parent_table, REFERENCED_COLUMN_NAME AS parent_column
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'LibraryManagementSystem' AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
```

Output: (9 rows)

child_table	fk_column	parent_table	parent_column
Books	AuthorID	Authors	AuthorID
Books	CategoryID	Categories	CategoryID
Books	PublisherID	Publishers	PublisherID
Fines	LoanID	Loans	LoanID
Loans	BookID	Books	BookID
Loans	MemberID	Members	MemberID
Loans	StaffID	Staff	StaffID
Reservations	BookID	Books	BookID
Reservations	MemberID	Members	MemberID

Explanation: The data dictionary lists nine foreign-key relationships, matching the cardinality table in Section 3.

Step 5: full definition of the Loans table (the table with the most relationships)

```
SHOW CREATE TABLE Loans;
```

Output: (1 row)

Table	Create Table
Loans	CREATE TABLE `Loans` (`LoanID` int(11) NOT NULL, `BookID` int(11) DEFAULT NULL, `MemberID` int(11) DEFAULT NULL, `StaffID` int(11) DEFAULT NULL, `IssueDate` date NOT NULL, `DueDate` date NOT NULL, `ReturnDate` date DEFAULT NULL, `Status` varchar(20) DEFAULT NULL, PRIMARY KEY (`LoanID`), KEY `BookID` (`BookID`), KEY `MemberID` (`MemberID`), KEY `StaffID` (`StaffID`), CONSTRAINT `Loans_ibfk_1` FOREIGN KEY (`BookID`) REFERENCES `Books` (`BookID`), CONSTRAINT `Loans_ibfk_2` FOREIGN KEY (`MemberID`) REFERENCES `Members` (`MemberID`), CONSTRAINT `Loans_ibfk_3` FOREIGN KEY (`StaffID`) REFERENCES `Staff` (`StaffID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

Explanation: Loans carries three foreign keys (Books, Members, Staff). MySQL automatically named them Loans_ibfk_1 to Loans_ibfk_3 and created an index on each foreign-key column. Assignment 6 replaces these with explicitly named constraints.

7. Design Notes

- **Surrogate integer keys:** every table has an INT primary key, which is compact, immutable and fast to join on.
- **NOT NULL** is applied to attributes that must always exist (names, issue/due dates, copy counts, fine amount).
- **UNIQUE** on *Books.ISBN* and *Members.Email* prevents duplicate catalogue entries and duplicate accounts; *Fines.LoanID* is UNIQUE so a loan can have only one fine.
- **DECIMAL(6,2)** is used for *Fines.Amount* because money must be stored exactly (no floating-point rounding).
- **VARCHAR for phone numbers** because they are identifiers, not quantities: they may contain leading zeros, "+" or hyphens.
- **Stricter rules** (CHECK, DEFAULT, referential actions) are intentionally left to Assignment 6, which adds them with ALTER TABLE.